

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This system analysis document introduces the strategic and technical foundation for Yama-inventory. This system is designed to solve manual, disjointed supply chain management issues into an automatic flow where the AI can communicate with other departments via emails to make restocking decisions based on any scenario based on the company's preferences.

1.1. Purpose:

So, the key element of system that has been covered are as followed:

1. **Architecture:** *A client-server architecture with a React-based frontend dashboard, a FastAPI backend serving REST and WebSocket endpoints, and an autonomous AI orchestration loop.*
2. **Data Flows:** *How supply chain events (e.g., unread supplier emails, sales orders) flow through the backend, are enriched by internal data repositories, analyzed by the ILMU AI model before making decisions which are displayed to the user via WebSockets.*
3. **Model Process:** *The end-to-end workflow of the agentic loop, which involves reasoning over tool schemas, requesting data from internal departments, checking for cross-SKU conflicts, and outputting structured JSON decisions (PO/RFQ drafts).*
4. **Role of Reference:** *This document serves as a high-level reference guide for developers, QA testers, and stakeholders involved in maintaining and scaling the platform.*

1.2. Background:

The company operates a complex supply chain involving multiple raw materials (e.g., Raw Steel Component X, Aluminum Casting Y) and finished SKUs. Previously, the coordination between sales orders, inventory levels, financial budgets, and supplier communications was manual and siloed, leading to stockouts, cross-SKU conflicts, and delayed responses to critical supplier alerts (e.g., shipment delays or price hikes).

1. Previous Version:

- a. *The existing processes relied on isolated CSV/spreadsheet tracking and manual email monitoring. The "previous version" of the workflow involved a human operator manually cross-referencing sales orders with inventory logs, reading supplier emails one-by-one, and manually drafting Purchase Orders (POs) and Request for Quotes (RFQs). The new capabilities introduced in Z.AI feature a centralized dashboard, continuous background email polling, and an AI agent that automatically detects issues and performs cross-department data aggregation.*

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Supply Chain Manager / Procurement	<i>Reviews AI-generated inventory decisions, monitor workflow alerts to solve, ensuring production workflow is smooth.</i>	<i>A unified dashboard displaying all actions taken by the AI, as well as their reasoning as well as setting preferences for AI to follow</i>
Sales Team	<i>Generates sales orders that dictate upcoming demand for finished goods.</i>	<i>Assurance that raw materials will be available to fulfill their sales orders on time without cross-SKU conflicts. Will send follow up email if lacking information.</i>
Suppliers (Vendors)	<i>Receive Request for Quotes (RFQs) and Purchase Orders (POs) generated by the system.</i>	<i>Timely, clear, and standardized communication regarding new orders or inquiries.</i>
Development Team	<i>Builds and maintains the Z.AI platform and its micro-components.</i>	<i>Clear API usage contracts, modular architecture (e.g., Dependency Inversion for the LLM Client), and well-documented agentic flows.</i>
QA Team	<i>Validates system behavior, API endpoints, and AI reasoning boundaries.</i>	<i>Defined mock scenarios, clear JSON schema constraints for the AI, and test coverage for the background polling loop.</i>

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Dashboard (React, Vite)
Architecture	Client-Server API with Agentic Background Loop

YamalInventory or Y-Inventory is structured as a client-server-based system. The client is a React Web Dashboard used by Supply Chain Managers. The dashboard communicates with a FastAPI Backend via RESTful APIs (for current status/data) and WebSockets (for real-time AI reasoning streams).

The backend architecture consists of:

- 1. Data Repository: Currently utilizing CSV files to simulate department databases (Inventory, Sales, Finance, Logistics, Manufacturing).*
- 2. Email Poller: An asynchronous background loop that polls an IMAP mailbox every 10 minutes for unread supplier emails.*
- 3. Agentic Orchestrator: The core intelligence that uses the ILMUClient to analyze input invoke tools (e.g., `get_sales_data`), generate candidate decisions and execute the most optimal decision`.*

2.1.2 LLM as Service Layer

The ILMU LLM is integrated as the core reasoning engine, heavily leveraging OpenAI SDK compatibility.

Dependency Diagram & Flow:

- Prompt Construction: The Orchestrator constructs a system prompt demanding JSON adherence and checks for cross-SKU conflicts. It passes user input and a suite of ToolRegistry schemas to the ILMU API.*
- Context Window Management: To enforce token limits and prevent context overflow, email bodies are explicitly truncated before being passed to the AI (body[:2000]). The agentic loop is strictly capped at MAX_STEPS = 10 to prevent runaway API usage.*
- Response Parsing: The backend receives the ChatCompletionMessage. If .tool_calls are present, the backend automatically maps the function names to the ToolRegistry, executes the local python functions (e.g., `get_inventory_data()`), appends the JSON results to the message history, and re-prompts the LLM.*

- *Final Delivery*: When no tool calls are requested, the final string is cleaned of markdown fences (e.g., ``json``, parsed as a JSON object, and streamed to the frontend via WebSocket.

2.1.3 Sequence Diagram

- Trigger**: The codebase is triggered by a new email -> Regularly monitor email inbox every 10 mins
- WebSocket Init**: Frontend establishes a WebSocket connection (/ws/orchestrator) and sends the payload.
- Agent Step 1**: Backend sends the prompt and tool schemas to ILMU API.
- Tool Invocation**: ILMU API returns a request to call `get_inventory_data` and `get_sales_data`.
- Local Execution**: Backend executes these tools against the Data Repository and returns the tabular data to ILMU.
- Final Decision**: ILMU analyzes the cross-SKU conflicts, decides to order 500 units of RAW-001, and sends the email to the supplier.
- Delivery**: Backend parses the JSON and pushes the result event to the frontend, displaying the Draft PO and decision taken. -> display action status

2.2. Technological Stack

- 2.2.1. **Frontend**: React 19, Vite, Recharts, React Router — Web-based dashboard for data visualization and AI chat interaction.
- 2.2.2. **Backend**: FastAPI, Uvicorn, asyncio, pandas — RESTful endpoints and WebSocket server handling the agentic loop and background tasks.
- 2.2.3. **AI Engine**: ILMU API (YTL AI Labs) — Handling prompt inference, multi-step tool calling, and text-to-JSON structuring. Accessed via the openAI Python SDK.
- 2.2.4. **Database / Storage**: Local CSV Datasets (Simulating internal PostgreSQL/ERP databases) and in-memory lists for active email alerts.
- 2.2.5. **External Integrations**: IMAP Server (for ingesting supply chain emails).

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD) Narrative:

1. **Email Ingestion**: External emails arrive at the Inbox. The `email_reader.py` background task fetches UNSEEN emails via IMAP.
2. **Email AI Classification**: The email body (truncated) is sent to ILMU (`ilmu-glm-5.1` model) to determine `is_issue` (e.g., shipment delay). If true, it is stored in the in-memory alert list.
3. **Data Repository**: Core department data (Inventory, Sales, Finance, Logistics) is read from CSV files using pandas,

4. **Orchestrator Loop:** Data moves from the CSVs -> Python Tools -> ILMU API -> JSON Response -> WebSocket -> React Dashboard table that is resolving the M: N relationship between orders and menu items.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
1	Action command centre	Users can view past and current action taken by AI and the explanation for the decision taken
2	Automated information handling	Background service fetches unread supplier emails, classifies them using AI, and automatically follows up with sender for missing data
3	Agentic Decision Engine	Users can trigger the Z.AI agent to reason over department data, identify cross-SKU resource conflicts, and make quantifiable reorder decisions. + company preferences and guardrails
4	Automated Draft Generation	Upon reaching a decision, the system automatically generates draft Purchase Orders, Work Orders, and Supplier RFQ Emails and sends them to corresponding departments.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	Requirements	Implementation
Scalability	The backend must handle concurrent WebSocket connections from multiple Supply Chain Managers.	Use of asyncio in FastAPI endpoints and threading (asyncio.to.thread()) for blocking IMAP network calls.

Reliability	<i>The agent must not fall into an infinite loop of API calls if it gets confused by data.</i>	<i>Strict enforcement of MAX_STEPS = 10 in the Orchestrator class. Graceful fallback to Mock reasoning if the API key is missing.</i>
Maintainability	<i>The codebase must allow easy swapping of LLM providers or addition of new tools.</i>	<i>Implemented using Dependency Inversion. The orchestrator depends on the LLMClient interface and a ToolRegistry abstraction.</i>
Token Latency	<i>LLM responses should not block the user interface entirely while "thinking".</i>	<i>WebSockets stream the internal "Reasoning / Calling Tool..." status steps to the UI so the user knows progress is happening.</i>
Cost Efficiency	<i>Prevent massive token expenditures on long, irrelevant promotional supplier emails.</i>	<i>Input text bodies are strictly truncate body[:2000] before classification, and a fast model (ilmu-glm-5.1 & gemini-pro-3,1) is used for email triage while the main orchestrator uses (ilmu-1.0).</i>

3.3. Out of Scope / Future Enhancements

- Direct ERP Database Integration:** Replacing the mock CSV Data Repository with live connectors to SAP/Oracle databases via robust ETL pipelines. It could also access reports on arriving supplies and production progress to update the warehouse database.
- Predictive Demand Forecasting:** Expanding the AI to run time-series forecasting (e.g., using ARIMA or specialized ML models) on historical sales data to predict future seasonality, rather than just reacting to current pending orders.
- Tracking of purchase orders :** In future we will implement purchase orders from others and whenever it reaches and a report is submitted, the report will be analyzed to update the database quick and ensure smooth workflow of manufacturing

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

- 4.1.1. **Grayscale Rollout & A/B Testing:** The system utilizes a modular tool registry. New tools (e.g. `get_predictive_forecast`) can be introduced for 5% of reasoning flows to evaluate if they improve the accuracy of the PO recommendations before full rollout.
- 4.1.2. **Emergency Rollback (ER) & Golden Release:** If the ILMU API experiences severe degradation, the system is designed to gracefully fallback to either rule-based keyword detection (for emails) or a Mock reasoning flow (for the orchestrator), ensuring the dashboard does not completely crash.
- 4.1.3. **Priority Matrix:**
 - 4.1.3.1. **P1 Critical:** Complete failure of the IMAP poller or ILMU API authentication. Action: System throws visual error flags; fallback logic engages; alert devops.
 - 4.1.3.2. **P2 High:** JSON parsing failures from the LLM. Action: The system strips markdown fences. If it still fails, the error is streamed to the user and the agent step halts safely.

4.2. Monitoring:

- 4.2.1. Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.
 - 4.2.1.1. Since our system depends on our brains ([Z.AI](#)) it all depends on the performance on the server. However we need to make our own precautions in case of a failure. Thus to overcome this, whenever an error regarding AI occurs, we will use other AI that have high availability servers with minimal downtime.

4.3. Assumptions: State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.

- a. **Universal Enterprise Modeling (CSV & Email Assumption):** We assume that every target company utilizes a completely different, often highly customized mix of legacy ERPs (SAP, Oracle, NetSuite) and communication channels. To make our hackathon prototype universally applicable without relying on proprietary API access, we use local CSV files to model the structured ERP database and automated Email generation to model the unstructured external communication channels.
- b. **Autonomous Execution & Guardrail Constraints:** We assume the system is trusted to act autonomously based on pre-set "Company DNA." The agent will automatically reallocate inventory and issue external Purchase Orders without human review. The constraint is that it operates strictly within user-defined financial and confidence guardrails (e.g., auto-spend limits). Human intervention is only required if a proposed action breaches these hard constraints.

- c. *LLM Cost Constraint: Estimated token usage per orchestration session is approximately 8,000–15,000 tokens (input + output across all tool calls). At current LLM pricing, this is roughly \$0.024–\$0.045 per session. To keep costs manageable, tool responses are pre-filtered server-side to only the relevant time window, passing the full unfiltered CSV to the model on every call would increase input tokens by 3–4x. Prompt caching is also applicable for repeated tool schemas.*
- d. *Technical Constraints: The system currently runs locally (FastAPI + React on localhost). No cloud deployment, production database (SQL/NoSQL), or authentication layer is included in this prototype scope.*
- e. *Performance Constraints: End-to-end agentic orchestration time is roughly 15–45 seconds depending on LLM API response latency, the complexity of the multi-variate math, and the number of tool calls required. This latency is communicated to the user via streaming status messages in the Control Center. No generation limit is enforced per session.*
- f. *Stateless Communication (No Historical Dependency): We assume that every incoming email represents an independent, self-contained event. New emails do not depend on the context of older emails to be understood or acted upon. Because of this assumption, the system processes communications statelessly—once an email triggers an action and the database is updated, the email's historical context is no longer required for future LLM reasoning. This eliminates the need to engineer complex context-retention or lifecycle-deletion algorithms for the prototype.*
- g. *Trusted Sender Environment: Due to the strict time constraints of the hackathon, we assume that all ingested emails originate from verified, legitimate actors (e.g., actual suppliers, internal personnel, or trusted clients). The system implicitly trusts the sender's identity and the data payload. Implementing robust zero-trust email security, phishing detection, or spam filtering is outside the scope of this proof-of-concept.*

4.4. External Dependencies: Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks & Mitigation
ILMU API (Z.AI)	The core LLM engine handling prompt inference, tool calling, and generating structured JSON output.	High Risk: Rate limits or API downtime. Mitigation: Fallback to <code>run_mock()</code> and <code>keyword fallback()</code> to ensure graceful degradation.

OpenAI Python SDK	Used as the communication client to interact with the ILMU API seamlessly.	Low Risk: Highly stable SDK.
IMAP Server	Used to fetch incoming emails from suppliers regarding delays/price changes.	Medium Risk: Connection timeouts. Mitigation: Wrapped in <code>try/except</code> and executed in a background thread to prevent blocking the main API thread.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

- 5.1.1. **Days 1-2 (Planning & Architecture):** Define the agentic loop, identify the cross-SKU conflict use case, structure the FastAPI + React template.
- 5.1.2. **Days 3-5 (Backend & AI Logic):** Implement the Dependency Inverted (LLMClient), the ToolRegistry, and the mock CSV DataRepository. Prompt engineering for strict JSON output.
- 5.1.3. **Days 6-8 (Frontend & Real-time Integration):** Build the React dashboard, hook up WebSocket streaming (/ws/orchestrator), and visualize the tables (Inventory, Sales).
- 5.1.4. **Days 9-10 (Refinement & Polling):** Add the asynchronous IMAP email polling task, refine fallback behaviors, and finalize documentation.

5.2. Team Members Role:

- 5.2.1. **Darren Teoh Pak Hui :** Responsible for backend logic and documentation
- 5.2.2. **Chang Juan Jue :** Responsible for backend, and documentation
- 5.2.3. **Teh Ern Lynn :** Responsible for frontend displays, backend and unit test draft
- 5.2.4. **Wong Yu Xuan :** Responsible for video editing, slide creating, script drafting and camera person

5.3. Recommendations:

- 5.3.1. **Transition from local CSV data parsing** (which loads the file on every API call) to a caching layer (like Redis) or an actual indexed Relational Database (PostgreSQL) to handle much larger manufacturing datasets efficiently.
- 5.3.2. **Distributed Task Queue:** Use Celery with Redis. Instead of a simple loop in the background, this allows you to scale the number of "AI Workers" horizontally across multiple servers.
- 5.3.3. **Distributed Locking:** Replace the local `threading.Lock()` with a Redis lock. This ensures that even if you run multiple instances of your backend, they won't try to process the same email at the same time.

- 5.3.4. **OAuth2 Integration:** Move away from storing email passwords. Use OAuth2 (Microsoft Graph or Google API) for more secure and professional email access.
- 5.3.5. **Observability:** Implement OpenTelemetry and structured logging. In production, you need to be able to trace exactly why an AI made a specific decision across the entire pipeline.
- 5.3.6. **Human-in-the-Loop (HITL):** Add an approval step in your dashboard for high-value decisions (e.g., any Purchase Order over \$5,000) before the AI actually sends the email.